

All of the metrics are really easy to read, and you can see how they change over time. Figure 8.11 has an interesting anomaly; over the last 30 days no code has changed but the number of issues and estimated tech debt have decreased. That's probably because this team ran their analysis and then changed the rule set by which the issues were measured. Updating code standards is a great task for any development team because it forces the team to look at the rules, discuss them, and decide what makes the most sense for the way they write code.

A metric that I'm not diving into is estimated technical debt. SonarQube has built-in algorithms that will calculate how long it should take your team to fix all the issues identified by static analysis. Even though this is a really cool feature of SonarQube, it doesn't take into account bigger pictures like potential problems with your build system or architectural issues related to integration between code modules.

Balancing tech debt and delivery